

An Orchestra of Agents

What I learned running a multi-agent system for
5,000+ developers

Muhammad Ahsan Ayaz · GDE in AI & Angular

AGNTCon + MCPCon Europe · Amsterdam · 2026

Scan for Slides & Code



- All links related to this session
- The postmortems I'll be quoting
- My socials

```
[2026-05-23 19:55] user report:
```

```
Q: "any good repos for Google's AntigraVity CLI?"
```

```
A: "dev.to temporarily unavailable."
```

5,000 developers. 16 agents. One **break** statement.

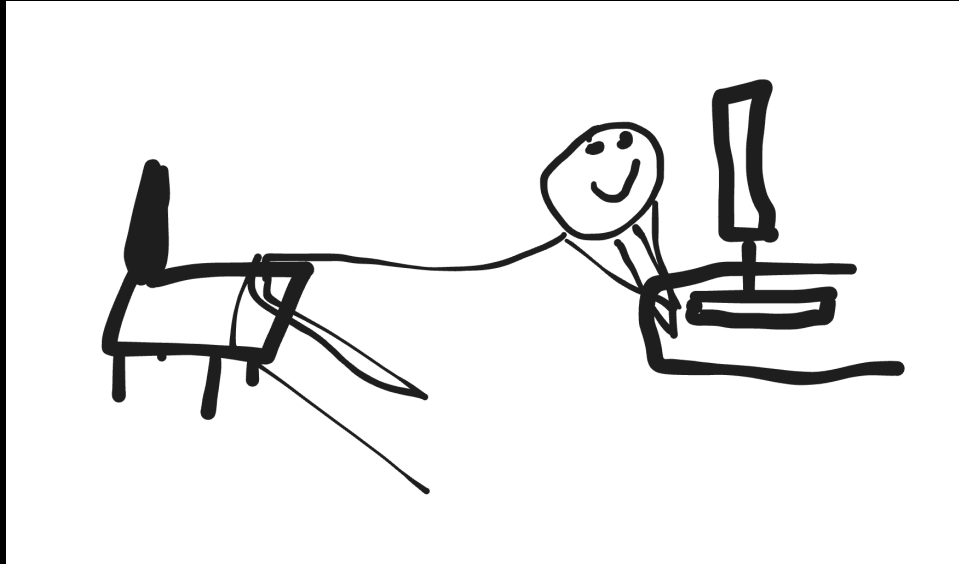
We'll come back to this.

Who here runs an AI agent in production?



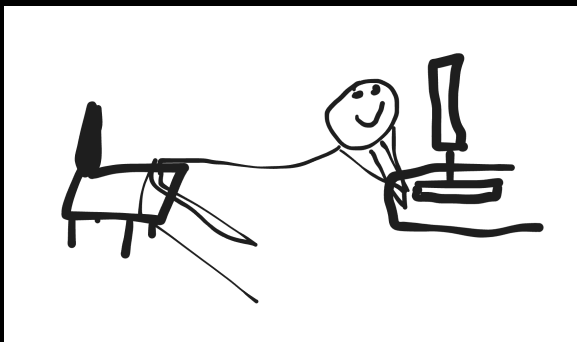
Who here runs more than one... talking to each other? 🙋

And who found their worst agent bug from a user's message instead of their telemetry? 🙋



prompt: Generate a super realistic image of a programmer using this drawing. Keep the weird pose as much as realistically possible.





+

[Prompt]

+









Whoami

Muhammad Ahsan Ayaz

- GDE in AI & Angular
- Software Architect
- 4x Author, 14M+ OSS installs
- Runs codewithahsan.dev (5,000+ devs)

Also me: shipped a `break` statement to 5,000 people 🤪

The problem with being the community

"How do I start with Angular? Any recent articles worth reading? Also... is anyone here open to mentoring me?"

One Discord message. Three different specialists. One of me.

What could go wrong?



Orchestrators route. Leaves work.

One big agent

Every skill in one prompt
Every failure everywhere
Nothing measurable



An orchestra

Specialists with one job each
A conductor that only routes
Failures isolated per seat

One bot, sixteen agents (the abstract says twelve)

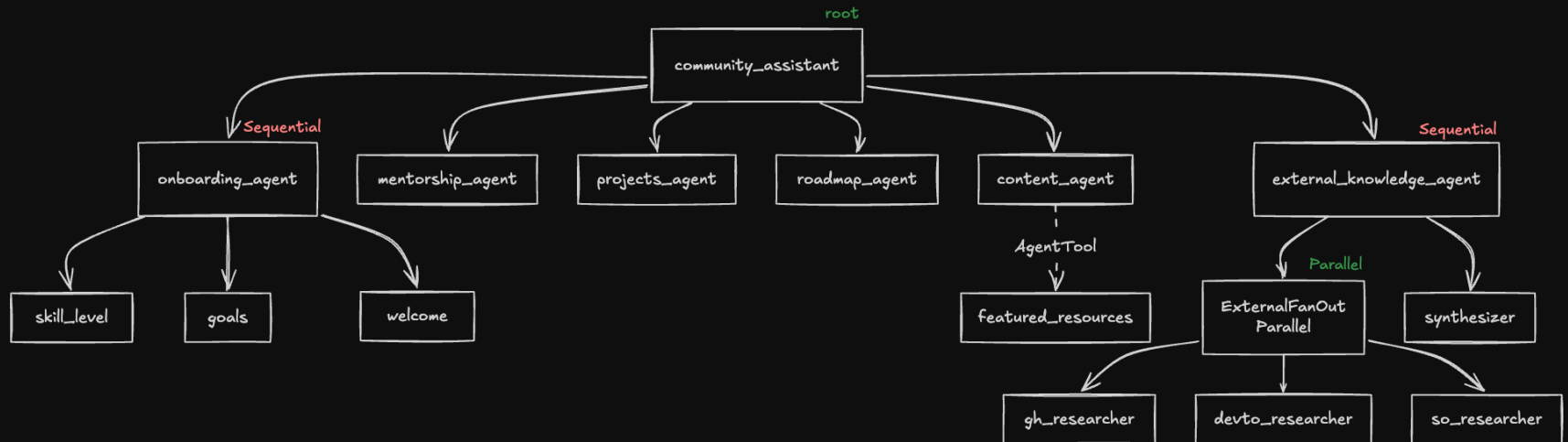
The abstract says 12.

My own docs say "12 total"... then list 13.

I counted for this talk. It's 16.



The tree



13 LlmAgents + 3 workflow agents. All **gemini-2.5-flash**. All in one process.

The root, in code

```
root_agent = LlmAgent(  
    name="community_assistant",  
    model=MODEL, # gemini-2.5-flash  
    instruction=ROOT_INSTRUCTION,  
    before_model_callback=[pii_sanitizer, inject_current_date],  
    sub_agents=[ # <-- this list IS the routing table  
        onboarding_agent, mentorship_agent, projects_agent,  
        roadmap_agent, content_agent, external_knowledge_agent,  
    ],  
)
```

sub_agents gives you LLM-driven routing for free. Each child's **description** is its routing advertisement.

The busy conductor pitfall

The orchestrator routes. It does not answer.

```
# BAD: root with tools and opinions of its own
root_agent = LlmAgent(
    tools=[search_blog_posts], # <-- root hoards the turn
    sub_agents=[content_agent], # ...and content_agent starves
)

# GOOD: route-only root, all work lives in leaves
root_agent = LlmAgent(
    instruction=ROUTING_INSTRUCTION,
    sub_agents=[content_agent, mentorship_agent],
)
```

Workflow agents: when you stop trusting the LLM

```
onboarding_agent = SequentialAgent(  
    name="onboarding_agent",  
    sub_agents=[  
        skill_level_extractor, # output_key="user_skill_level"  
        goals_extractor,      # output_key="user_goals"  
        welcome_agent,        # user-facing, runs LAST on purpose  
    ],  
)
```

output_key is scalar: one agent, one state key. Two facts = two silent extractors. State is the contract.

The fan-out

```
external_knowledge_fan_out = ParallelAgent(  
    name="ExternalFanOut",  
    sub_agents=[gh_researcher, devto_researcher, so_researcher],  
)  
  
external_knowledge_agent = SequentialAgent(  
    name="external_knowledge_agent", # routable name on the WRAPPER  
    sub_agents=[external_knowledge_fan_out, synthesizer],  
)
```

GitHub, dev.to, Stack Overflow in parallel. Then one synthesizer voice.

The parallel win

	time
gh_researcher	4,463 ms
devto_researcher	4,570 ms
so_researcher	5,251 ms
Parallel total	5,251 ms
Serial would be	14,284 ms

The bottleneck didn't disappear. It moved.

End-to-end turn: 17.6 s

The synthesizer alone: 10.9 s



Boring tools that never miss

```
def search_github(query: str) -> dict:
    """Search GitHub repositories for a topic.""" # docstring IS the schema
    try:
        resp = _client.get("/search/repositories", ...) # 10s timeout
        ...
    except httpx.HTTPError:
        return {"status": "error", "detail": "github unavailable"}
    # <-- return, never raise
```

Plain Python functions. ADK derives the tool schema from signature + docstring.

So. How much MCP holds this together?

Zero.



Why function tools won here

- One process. I own **both ends** of every call
- Typed dict in, typed dict out... no serialization boundary
- No server lifecycle, no auth ceremony, no extra hop to monitor

Where MCP earns its place

- Tools that cross an **org boundary**: someone else's system, someone else's auth
- Tools consumed by **clients you don't control**: Claude, IDEs, other people's agents
- Dev-time tooling, where the server ecosystem is genuinely a gift

Callbacks: the immune system

PII sanitizer

Redacts before the model sees it.

Tool cache

600 s TTL on repeat searches.

Lifecycle telemetry

JSON to stdout. Zero infra.

Date injection

The model learns what "today" is.

May 2026. The bot recommends...

"Front-end Weekly News Week - 18"

published 2020



The fix is ten tokens

```
def inject_current_date(callback_context, llm_request):  
    today = datetime.now(timezone.utc).strftime("%Y-%m-%d")  
    llm_request.append_instructions([  
        f"Today is {today} (UTC). When surfacing third-party "  
        f"content, prefer the most recent items unless the user "  
        f"asks for historical material."  
    ]) # <-- system_instruction, not a get_date TOOL (that costs a round trip)
```

Verified with a query about a tool announced 2 days earlier: all 5 repos dated inside those 2 days.

A websocket bot on serverless (yes, really)

- `--min-instances=1` : scale-to-zero hangs up the Discord websocket
- `--max-instances=1` : in-memory sessions... two instances = split-brain memory
- `--no-cpu-throttling` : Cloud Run throttles CPU between requests. A websocket bot's work happens between requests.
- `PYTHONUNBUFFERED=1` : or the stdout telemetry pipeline just... buffers

The epilogue

Always-on + unthrottled CPU = **instance-based billing**

1 vCPU rented 24/7 $\approx$ **\$47 / month**. The free tier covers ~50 hours of it.

Today the same container runs on an Always-Free **e2-micro** VM:
~\$3-4 / month

Same Dockerfile, same image. The flags that keep a websocket bot alive on Cloud Run are the flags that make it expensive there.

So: sanitized, cached, instrumented,
deployed.

Nothing could go wrong. Right?

The night the telemetry lied

[2026-05-23 19:55] user report:

Q: "any good repos for Google's Antigravity CLI?"

A: "dev.to temporarily unavailable."

The loop that looked correct

```
async for event in runner.run_async(...):  
    events_seen.append(event)  
    if event.is_final_response() and event.content:  
        response_text = event.content.parts[0].text or ""  
        break # <-- the entire incident
```

Reads perfectly. Reviewed. Shipped. Wrong.

The docstring I didn't read slowly enough

"...when multiple agents participate in one invocation, there could be one event has `is_final_response()` as True for each participating agent."

ADK's own `Event.is_final_response` docs.

The fastest agent wins. The fastest agent was broken.

- dev.to 404s on the two-day-old tag → error dict → done in 1.2 s
- GitHub: 2.4 s. Stack Overflow: 2.6 s. The synthesizer: never heard from
- My "graceful degradation" is now the bot's answer

And then the **break** pulled the pin

break → **GeneratorExit** → **asyncio.TaskGroup** cancelled
mid-flight

Root + synthesizer exit callbacks: never fired

The bug corrupted the telemetry built to catch it.

How long? Since the fan-out shipped.

The synthesizer's reply had never once reached a user.

*"Local **adk web** drains the **entire** event stream, so synthesizer output always rendered. The smoke test was a false positive."*

The dev tool was more forgiving than production.

37 minutes. The fix: drain everything, keep the last word.

```
async for event in runner.run_async(...):
    events_seen.append(event)
    if event.is_final_response() and event.content and event.content.parts:
        text = event.content.parts[0].text or ""
        if text:
            response_text = text # keep latest; synthesizer wins
# no break. The stream ends when the stream ends.
```

Report 19:55 → root cause 20:01 → deployed 20:32. Leaves speak first, orchestrators speak last.

The clean trace

agent.enter	gh_researcher		20:33:43.220
agent.enter	devto_researcher		20:33:43.220
agent.enter	so_researcher		20:33:43.221
agent.exit	devto_researcher	1365ms	20:33:44.586
agent.exit	so_researcher	5009ms	20:33:48.229
agent.exit	gh_researcher	7025ms	20:33:50.246
agent.enter	external_knowledge_synthesizer		20:33:50.248
agent.exit	external_knowledge_synthesizer	3637ms	20:33:53.883
agent.exit	community_assistant	13072ms	20:33:53.884

Three leaves entering within 1 ms. Every enter pairs with an exit. This is what healthy looks like.

The confession, and the fix for the fixer

That night, I never wrote the regression test.

```
async def test_synthesizer_beats_fastest_leaf_in_fan_out_race():
    events = [
        _final("devto_researcher", "dev.to temporarily unavailable."),
        _final("gh_researcher", "Found 5 GitHub repos."),
        _final("so_researcher", "Found 4 Stack Overflow questions."),
        _final("external_knowledge_synthesizer", synthesizer_answer),
    ]
    text = await drain_final_response(_stream(events, []), events_seen=[])
    assert text == synthesizer_answer
```

Written while preparing this talk. It passes. It would have failed for weeks.

Event5 of 16

EventRequestResponse


```
content:
  parts: Array[1] [{"functionCall":{"id":"adk-6982a028-0452-4144-b4
    role: "model"
  finishReason: "STOP"
  usageMetadata:
    candidatesTokenCount: 23
    promptTokenCount: 1099
    promptTokensDetails:
      0:
        modality: "TEXT"
        tokenCount: 1099
    thoughtsTokenCount: 178
    totalTokenCount: 1300
  invocationId: "e-263ea6cc-bf09-4997-9cab-ac3d345ebfcf"
  author: "projects_agent"
  actions:
    stateDelta:
    artifactDelta:
    requestedAuthConfigs:
    requestedToolConfirmations:
  longRunningToolIds:
    id: "ca3700d7-66d1-4344-8e86-4d8f2b22b97e"
    timestamp: 1789593449.820582
    title: "functionCall:transfer_to_agent"
```

SESSION ID de2963a3-138a-4ce6-add5-267b5fdee17d

Token Streaming | + New Session

Can you find the code-with-ahsan github repo?

transfer_to_agent

transfer_to_agent

transfer_to_agent

transfer_to_agent

I can't help you find GitHub repos, as I can only search for articles on devto.

I can only search Stack Overflow for answered questions, not GitHub repositories. I recommend trying a different search engine or a dedicated GitHub search.

search_github_repos

search_github_repos

Here's the top result for "code-with-ahsan" on GitHub:
AhsanAyaz/code-with-ahsan A repository for courses and tutorials on Next.js, React, and more. Stars: 27, Language: TypeScript, Last updated: 2026-09-12 URL: <https://github.com/AhsanAyaz/code-with-ahsan>

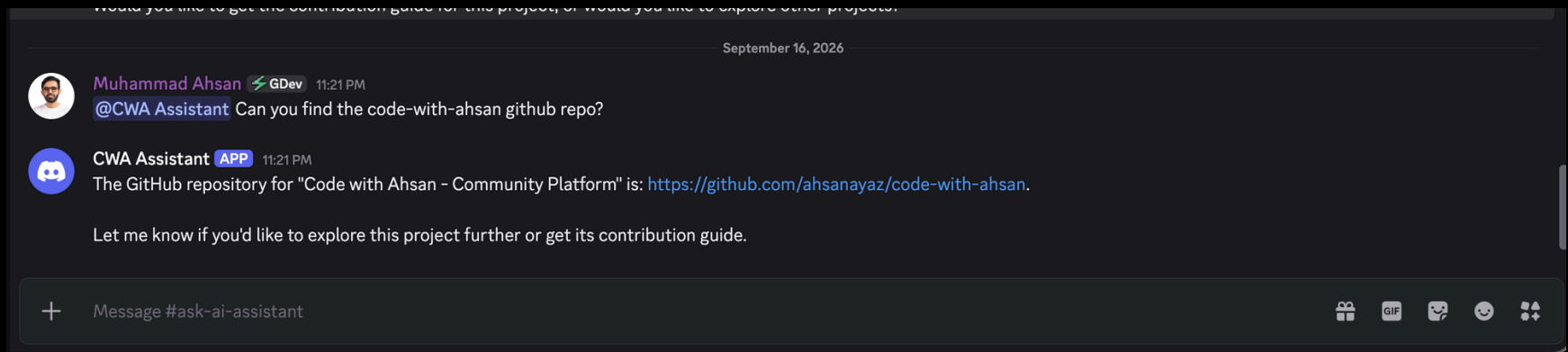
Here is the GitHub repository for "code-with-ahsan":
AhsanAyaz/code-with-ahsan A repository for courses and tutorials on Next.js, React, and more. Stars: 27 Language: TypeScript Last updated: 2026-09-12 URL: <https://github.com/AhsanAyaz/code-with-ahsan> (via GitHub)
The dev.to and Stack Overflow search functions are temporarily unreachable for GitHub repository searches.
Would you like to explore another topic related to Ahsan's work or ask a different question about this repository?

Type a Message...

Leaves don't get opinions about scope. That's the orchestrator's job... in both directions.

Captured last night, rehearsing this talk. Fix is still a proposal doc. Find me after and tell me where it's wrong.

What the user actually sees



Same night, same query, through the fixed drain loop: the synthesizer's answer, refusals swallowed.

Patterns cheat sheet

Pattern	When	The gotcha
LLM auto-transfer routing	intent is genuinely open	route-only root: no tools, no opinions
SequentialAgent + output_key	order must be guaranteed	keys are scalar; state is the contract
ParallelAgent fan-out	independent I/O branches	total = slowest branch + the merge
Function tools	you own both ends of the call	return error dicts, never raise in a branch
MCP	tools cross an org/client boundary	don't pay a server hop for your own code
Callbacks	cross-cutting concerns	fail open: try/except, return None
Cloud Run for a gateway bot	request-driven work only	always-on flags flip it to instance billing; a tiny VM wins

Pattern	When	The gotcha
Event-stream drain	any production runner	<code>is_final_response()</code> fires per agent

Resources & Code



The repo · the postmortems · the soak logs · these slides

The one thing to remember

Determinism where you can.
An LLM only where you must.
And drain the whole stream.

Orchestrators route. Leaves work. Production keeps the receipts.

Thank you, Amsterdam

Muhammad Ahsan Ayaz

 codewithahsan.dev

 [@codewith_ahsan](https://twitter.com/codewith_ahsan)

 linkedin.com/in/ahsanayaz

Code from today:
[github.com/AhsanAyaz/
code-with-ahsan](https://github.com/AhsanAyaz/code-with-ahsan)

Q&A - ask me anything
(especially about the unfixed bug)